

1. 今日は「DAO」を固定する（概要）

この回は、Java から DB に命令する作業を **DAO（DB担当窓口）** に集約し、上流側（呼び出し側）と切り離すための回です。**第3回は参照系（SELECT）のみ扱います。**

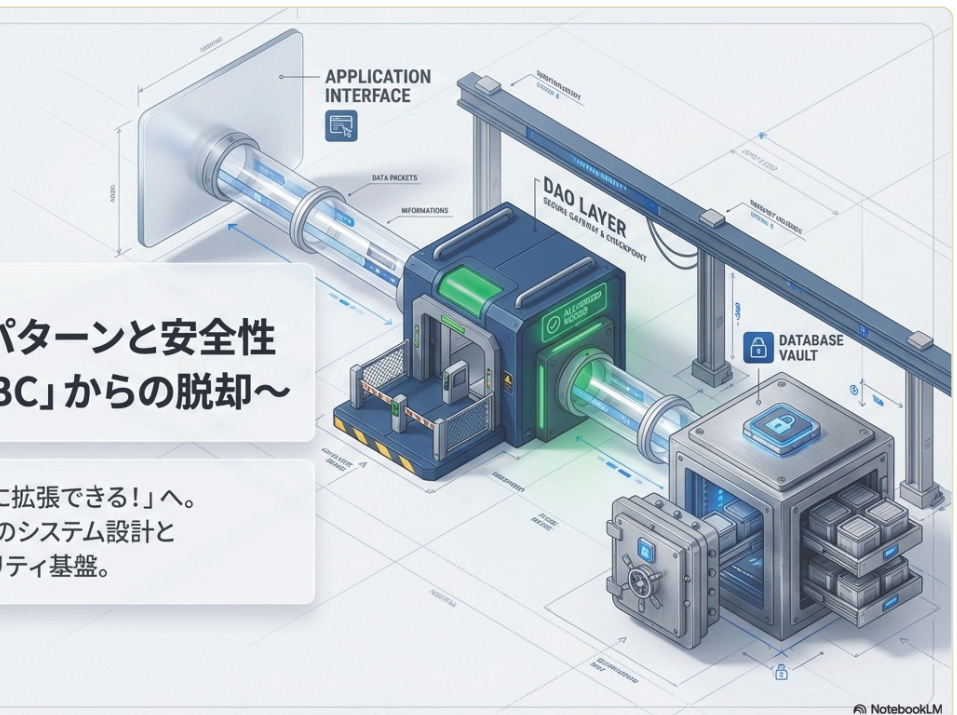
題材は **emps**（社員）と **depts**（部署）です（環境構築 HTML の **lesson DB**）。単表の検索から、2テーブル結合（JOIN）まで段階的に進めます。

INSERT / UPDATE / DELETE とトランザクションは第4回（Unit04_DAO_Update）で扱います。

Session 03

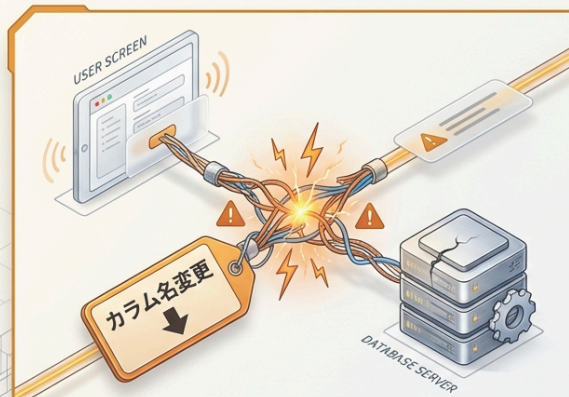
第3回：DAOパターンと安全性 ～「直書きJDBC」からの脱却～

「動いた!」から「安全に拡張できる!」へ。
エンタープライズ対応のシステム設計と
データベース・セキュリティ基盤。



2. なぜDAOが必要？（直書きJDBCの弱点）

動く＝正解ではない。「直書きJDBC」の2つの致命的弱点



変更に弱い (Brittle to Change)

DBのカラム名が1つ変わるだけで、プロジェクト中の全ファイルを検索し、SQLを書き直す地獄が待っている。



攻撃に弱い (Vulnerable to Attack)

悪意あるユーザーが入力欄に特殊な文字を入れただけで、データベースが破壊されたり、全情報が盗み出される。

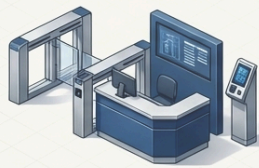
© NotebookLM

要点

- SQLやDB処理を呼び出し側に散らすと、修正のたびに影響範囲が増えやすい（変更に弱い）
- 文字列連結でSQLを組み立てると、意図しない入力混ざりやすい（攻撃に弱い）

3. DAO / DTO / Entity の3箱（確認）

重要用語：データを運ぶ「3つの箱」



[用語 / フルネーム]	DAO (Data Access Object)	(Entity) (Entity)	DTO (Data Transfer Object)
[役割]	DBの受付窓口。 SQLを唯一知っているクラス。	テーブルの写し鏡。 DBの1行と1対1で対応する Javaクラス。	データの運び屋。 画面に表示したい項目だけを 集めたクラス。
[物理メタファー]	厳重な搬入口と警備員	規格化された 倉庫のコンテナ	カスタマイズされた 配送用ダンボール

© NotebookLM

上の図は、上流から見たときの役割の分け方（DAO / DTO / Entity の3箱）を示す概念図です。この回のサンプルコードでは、Entity と DTO を同一視し、EmpDto（JavaBeans）1 クラスだけで進めます（部署名が要るときは deptName フィールドにだけ JOIN 結果を載せます）。

▼ 確認（Service → DAO → DB）

Service → DAO → DB
DTO = 運ぶ箱（画面/呼び出し向け）
Entity = DBの行（テーブル向け）

4. 学び方のコツ（最初は型を真似する）

最初は、動くコードの型をそのまま真似して動かし、「どこを変えると何が変わるか」を確認しながら理解を深めてください。下のサンプルは Eclipse などに貼り付けて動かす前提です。

5. DAOクラスの基本型（DaoBase・EmpDto・EmpDao）

（参考）Entity と DTO をクラスとして分ける場合

- **メリット**：例として、パスワードのハッシュなど、画面や公開 API に載せたくない列は、DB に近い側（いわゆる Entity 側）にだけ保持し、外に運ぶ DTO には含めない、といった切り分けがしやすい。
- **デメリット**：クラスやマッピング（詰め替え）が増え、名前の使い分けも悩みやすくなる。

大規模なアプリではこのように分けることがあります。が、**本講義規模の JDBC では次の EmpDto（JavaBeans）だけで十分です。**

- **接続情報**は設計にもよるが、ここでは親クラス **DaoBase** に `getConnection()` を用意して集約する。
- **テーブルごとに1クラス**：例として **emps** 用に **EmpDao** を用意する（部署マスタは必要に応じて別 DAO に分けてもよい）。
- **第3回（参照系）**：`findById` / `findAll` など、**SELECT** で **EmpDto**（またはリスト）を返す。
- **第4回（更新系）**：`insert` / `update` / `delete` とトランザクションは `Unit04_DAO_Update` で学ぶ。サンプルの `EmpDao.java` には第4回用メソッドも載せてあるが、今回の講義では読まなくてよい。

▼ DaoBase（ドラッグでコピー・クリックで開く）

マウスで全選択して貼り付け欄へドラッグしてください：

```
import java.sql.Connection;
import java.sql.DriverManager;
import java.sql.SQLException;

/**
 * DAO 共通クラス：接続情報を1か所に集約する。
 */
public class DaoBase {
```


▼ EmpDto (JavaBeans・ドラッグでコピー・クリックで開く)

fname = 苗字、lname = 名前（環境構築 SQL のコメントに合わせる）。deptName は単表 SELECT では null、JOIN するときだけセットする。

マウスで全選択して貼り付け欄へドラッグしてください：

```
/**
 * emps テーブルの1行を表す DTO（本講義では Entity と同一視）。
 * deptName は JOIN 時のみ使用（単表なら null）。
 */
public class EmpDto {

    private int empId;
    private String fname;
```

▼ EmpDao : 代表メソッド (findByld の完成例・第3回)

この教材では、DAO メソッドに throws SQLException を付けず、DAO 内で catch (SQLException e) して e.printStackTrace() する形を基本にします（本来はログ設計・通知・リカバリーを検討します）。例外の考え方は **第3回前に** Unit90_Exception で押さえておくと読みやすくなります。

マウスで全選択して貼り付け欄へドラッグしてください：

```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class EmpDao extends DaoBase {

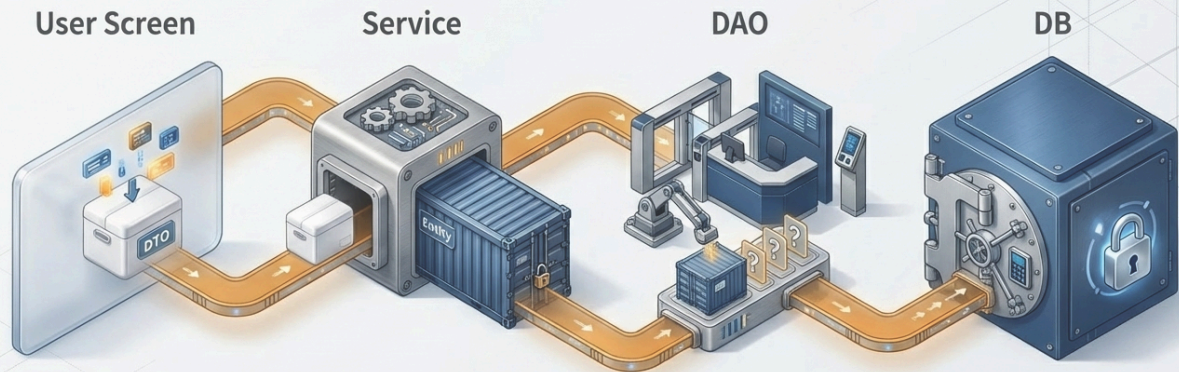
    /**
```

流れの確認と記述の型は、「6. データの旅」の直後の見出し 7（理解を深める問題）で、商品テーブル items と ItemDao を題材にまとめて行います（この節のコード例は emps のままです）。SQL の細かい穴埋めや JOIN (emps / depts) は 03-練習問題-DAO参照系 (emps-depts).html で扱います。

完全版のソースは **Unit03_DAO/src** (DaoBase.java / EmpDto.java / EmpDao.java)。参考として ItemDto.java / ItemDao.java (items 用) も同フォルダに残しています。

6. データの旅（保存・取得）

データの旅路：往路（保存するとき）



1. 呼び出し側（画面/Main）

画面からの入力を DTO に詰める。

2. Service（中継役）

必要なら計算などを行い、DB用の Entity へ変換して渡す。

3. DAO（窓口）

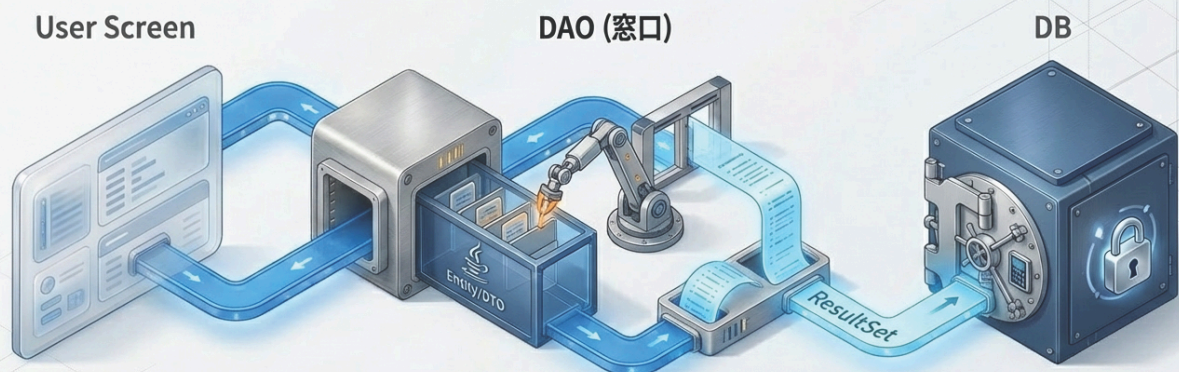
Entity から値を取り出し、PreparedStatement の「？」に埋めて SQL を発行する。

4. DB

データを安全に永続化（保存）する。

© NotebookLM

データの旅路：復路（取得するとき）



3. 呼び出し側

DAO から返ってきた安全な「Java のオブジェクト」を受け取って画面に表示する。

2. DAO（窓口）

ResultSet の各列を一行ずつ読み取り、Entity（または DTO）のフィールドに「写経」するように詰め替える。

1. DB

検索結果として生のデータセット（ResultSet）を返す。

© NotebookLM

ポイントは、**呼び出し側は「DAOに頼むだけ」**、DAOが PreparedStatement と ResultSet を責任を持って扱うことです。

7. 理解を深める問題（1本）～items / ItemDao の findById

テーマは **商品テーブル items** と **ItemDao の findById（1件検索）** に固定します。まず**流れの並べ替え**で更新系との違いを意識し、つづけて**記述**で型・コメント・パート別の穴埋めを身につけます。記述で詰まったら**ヒント①→②→③**の順に開いてください。

【流れの把握】並べ替え（紛らわしい文つき）

問題1 次の **ア～コ** のうち、**ItemDao#findById（参照系・1件取得）** として**実際に行う処理だけ**を取り出し、**実行される順番**に並べてください（例：ウ→ア→... のように記号の列をノートに書く）。**使わない文が4つ**残ります。さらに、残った4つを「更新系APIの誤用」「順序が不可能」「トランザクションの混同」のどれかに分類し、理由を一言で書いてください。

ア SQL 文字列を用意する（`SELECT item_id, category, item_name, unit_price FROM items WHERE item_id = ?`）

イ `getConnection()` でデータベースへの接続を取得する

ウ `prepareStatement(sql)` で SQL をドライバに渡し、実行準備（プリコンパイル）をする

エ `setInt(1, itemId)` でプレースホルダに検索キーを格納する

オ `executeQuery()` で検索を実行し、得られた `ResultSet` を開く

カ `rs.next()` でカーソルを進め、行があれば `getXxx` で列を読み `ItemDto` を組み立てて返す（なければ `null`）

キ 検索を `executeUpdate()` で実行し、その戻り値を `ResultSet` に格納する

ク `INSERT` 文を `executeUpdate` し、続けて同じ `ResultSet` で `SELECT` 結果を読む

ケ 接続を取得する**前に**、`ResultSet` のカーソルで `item_name` を読み取る

コ `commit()` でトランザクションを確定してから、SQL 文字列を用意する

▼ 自分の答え・貼り付け用（クリックで開く）

正しい順の記号列と、使わなかった記号とその理由をメモできます。

ここに並べた記号列を書く

問題2（確認） 呼び出し側に「items の1行分のデータ」を運ぶための箱として、まず連想するクラスはどれですか。

- **A** `DaoBase`
- **B** `ItemDao`
- **C** `ItemDto`

【記述①】クラス名・型の穴埋め

____ を埋めてください。

```
public class ____ extends ____ {  
  
    public ____ findById(int itemId) {  
        // 以降は記述②・③へ  
    }  
}
```

▼ 自分の答え・貼り付け用（クリックで開く）

穴埋め後のクラス宣言を書く

【記述②】コメントを簡潔で正確な日本語に書き換える

次の // （…） を、処理内容が伝わる**講義向けの短い日本語**に置き換えてください。解答例には**模範コメントの一覧とコメント付き全文**の両方があります。


```

public ItemDto findById(int itemId) {
    String sql = "SELECT item_id, category, item_name, unit_price FROM items WHERE item_

    // (ここ①: この try がまとめて行っていること)
    try (Connection con = getConnection();
        PreparedStatement ps = con.prepareStatement(sql)) {

        // (ここ②: SQL の「?」と対応づける操作)
        ps.setInt(1, itemId);

        // (ここ③: 内側の try が担うこと)
        try (ResultSet rs = ps.executeQuery()) {
            // (ここ④: ResultSet のカーソルと1行分のデータ)
            if (rs.next()) {
                return new ItemDto(
                    rs.getInt("item_id"),
                    rs.getString("category"),
                    rs.getString("item_name"),
                    rs.getInt("unit_price"));
            }
            // (ここ⑤: 該当行が無いとき)
            return null;
        }
    }
}

```

▼ 自分の答え・貼り付け用（クリックで開く）

コメントを書き換えたメソッド全体を貼る

【記述③】パート別の穴埋め → 全体をつなげて実行できる形に

①～⑤はそれぞれ ____ を埋めたうえで、最後に1つのメソッドとして連結できるか確認します（import とクラス宣言は解答例を参照）。

① データベースへの接続

```
try (Connection con = ____());
```

▼ 自分の答え・貼り付け用（クリックで開く）

① 埋めた行を書く

② SQL を準備（プリコンパイル）

```
PreparedStatement ps = con. ____ ( ____ ) {
```

▼ 自分の答え・貼り付け用（クリックで開く）

② 埋めた行を書く

③ プレースホルダに値を格納

```
ps. ____ ( ____ , itemId);
```

▼ 自分の答え・貼り付け用（クリックで開く）

③ 埋めた行を書く

④ SQL を実行して DB から結果セットを得る

```
try (ResultSet rs = ps. ____ ()) {
```

▼ 自分の答え・貼り付け用（クリックで開く）

④ 埋めた行を書く

⑤ ResultSet のカーソルでデータを取り出す

```

        if (rs.____()) {
            return new ItemDto(
                rs.getInt("____"),
                rs.getString("____"),
                rs.getString("____"),
                rs.getInt("____"));
        }
        return null;
    }
}

```

▼ 自分の答え・貼り付け用（クリックで開く）

⑤ 埋めたブロックを書く

全体：①～⑤をつなぎ、findById メソッドとして完成させる

▼ 自分の答え・貼り付け用（メソッド全体・クリックで開く）

findById の完成コード（import は別／クラス内メソッドのみでも可）

▼ ヒント①

並べ替えでは「参照は executeQuery」「更新件数は executeUpdate」を切り分けます。接続より先に ResultSet はあり得ません。

記述①は「誰が誰を継承し、何を返すメソッドか」。記述③の①～⑤は、接続→文の準備→プレースホルダー→実行→カーソル、の五段に対応します。

▼ ヒント②

外側の try-with-resources は `getConnection()` と `prepareStatement(sql)` をまとめて閉じます。

`setInt(1, itemId)` の `1` は SQL 内のプレースホルダの先頭です。

⑤の列名は SELECT 句と一致させます（スネークケースのまま文字列で指定）。

▼ ヒント③

記述① : `ItemDao`、`DaoBase`、戻り値型 `ItemDto`。

問題1 の正しい順 : **ア** → **イ** → **ウ** → **エ** → **オ** → **カ**。使わないのは **キ・ク・ケ・コ**（解答例に理由あり）。

記述③ : ① `getConnection` ② `prepareStatement, sql` ③ `setInt, 1` ④ `executeQuery` ⑤ `next` と列名 `item_id`, `category`, `item_name`, `unit_price`。

記述②の模範コメントは「解答例」内の箇条書きとコードを参照。

▼ 解答例

【解説・確認用】

問題1 正しい実行順は **ア → イ → ウ → エ → オ → カ**。**キ**は参照に `executeUpdate` は使わず、戻り値も `ResultSet` ではありません。**ク**は `INSERT` と `SELECT` が混ざった誤ったイメージです。**ケ**は接続前に `ResultSet` を読めません。**コ**は単純な1件取得では不要な順序で紛らわしいです。

問題2 正解は **C (ItemDto)**。`ItemDao` は窓口、`DaoBase` は接続の共通部です。

記述②の模範コメント（ここ①～⑤）

- ① 接続を取得し、実行する SQL 文をバインドする (try-with-resources)
- ② 先頭のプレースホルダに `itemId` をセットする
- ③ 検索を実行し、結果セット (カーソル) を開く
- ④ カーソルを1行進め、行があれば列から値を読み取り `ItemDto` を組み立てて返す
- ⑤ 該当行がなければ `null` を返す

記述①・②・③（コメント付きメソッド全文）

```
public ItemDto findById(int itemId) {
    String sql = "SELECT item_id, category, item_name, unit_price FROM items WHERE item_id = ?";

    // 接続を取得し、実行する SQL 文をバインドする (try-with-resources)
    try (Connection con = getConnection();
        PreparedStatement ps = con.prepareStatement(sql)) {

        // 先頭のプレースホルダに itemId をセットする
        ps.setInt(1, itemId);

        // 検索を実行し、結果セット (カーソル) を開く
        try (ResultSet rs = ps.executeQuery()) {
            // カーソルを1行進め、行があれば列から値を読み取り ItemDto を組み立てて返す
            if (rs.next()) {
                return new ItemDto(
                    rs.getInt("item_id"),
                    rs.getString("category"),
                    rs.getString("item_name"),
                    rs.getInt("unit_price"));
            }
            // 該当行がなければ null
            return null;
        }
    }
}
```

マウスで全選択して貼り付け欄へドラッグしてください（参考・メソッドのみ）：

```
public ItemDto findById(int itemId) {
    String sql = "SELECT item_id, category, item_name, unit_price FROM items
WHERE item_id = ?";

    // 接続を取得し、実行する SQL 文をバインドする (try-with-resources)
    try (Connection con = getConnection();
        PreparedStatement ps = con.prepareStatement(sql)) {
```

ItemDao クラス・import 付き（Unit03_DAO/src/ItemDao.java の findById 部と一致・ドラッグ用）：

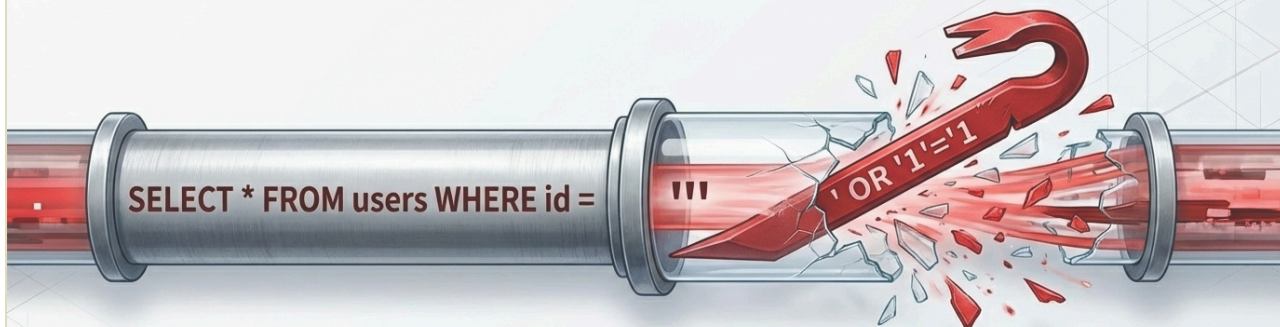
```
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;

public class ItemDao extends DaoBase {

    /**
```

8. PreparedStatement : ? と setXxx の対応

攻撃の解剖学：SQLインジェクション

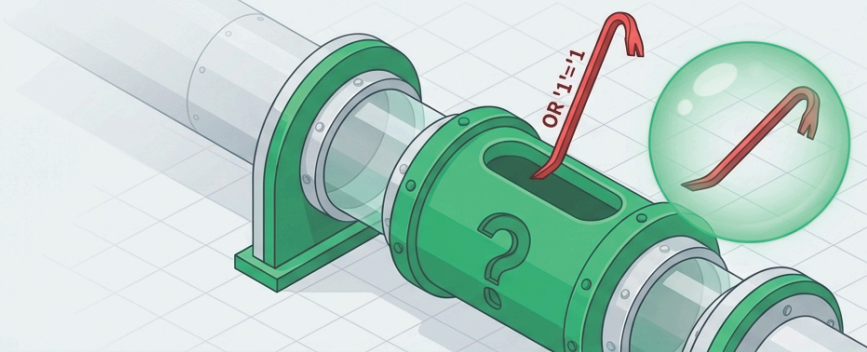


文字列連結 (+) の致命的な罠。

ダメな例: "SELECT * FROM users WHERE id = " + inputId + ""
もしユーザーが名前欄に ' OR '1'='1 と入力したら？
結果: SELECT * FROM users WHERE id = ' OR '1'='1'
SQLの構造そのものが破壊され、「常に真」になる命令に化ける。
すべてのユーザーデータが盗まれる。

© NotebookLM

最強の盾：PreparedStatementによる無効化



無害化（サニタイズ）のメカニズム

```
String sql = "SELECT * FROM users WHERE id = ?";
```

「？」は、あとから値を入れる「予約席」。

```
pstmt.setString(1, inputId);
```

この瞬間、どんな悪意ある文字が入ってきても、DB側では「単なるデータ（文字列）」として封じ込められる。
SQLの命令文として解釈される（書き換わる）心配は一切ない。

© NotebookLM

実装の比較：脆弱性の排除

⚠ 危険！絶対にやってはいけない書き方（文字列連結）

```
// 危険！絶対にやってはいけない書き方
String sql = "SELECT * FROM users WHERE id = '" + user_id + "'";
Statement stmt = con.createStatement();
ResultSet rs = stmt.executeQuery(sql);
```

🛡 安全！予約席(?)を使った書き方（PreparedStatement）

```
// 安全！予約席(?)を使った書き方
String sql = "SELECT * FROM users WHERE id = ?";
PreparedStatement pstmt = con.prepareStatement(sql);
pstmt.setString(1, user_id); // ?に安全に値をセット
ResultSet rs = pstmt.executeQuery();
```

NotebookLM

▼ 最小コードイメージ (emps 検索)

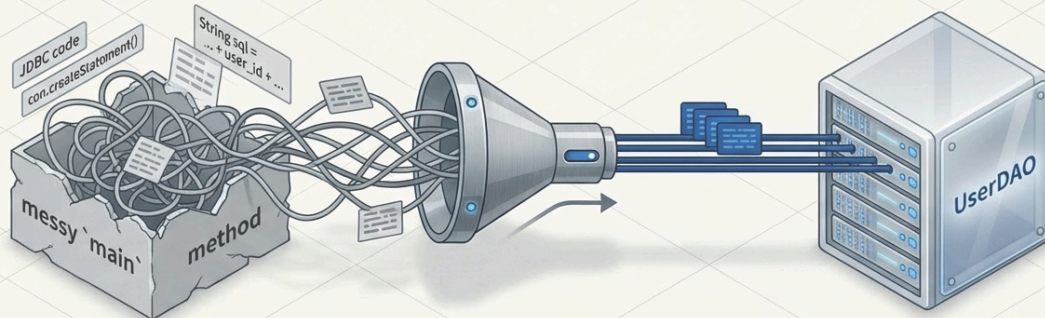
```
String sql = "SELECT fname, lname, salary FROM emps WHERE dept_no = ?";

try (Connection con = DriverManager.getConnection(URL, USER, PASS);
     PreparedStatement ps = con.prepareStatement(sql)) {
    ps.setInt(1, deptNo); // 先頭の「?」に値を入れる

    try (ResultSet rs = ps.executeQuery()) {
        while (rs.next()) {
            // rs.getString(...), rs.getInt(...)
        }
    }
}
```


9. 最小のミッション（DAOへ切り出す）

ミッション・ブリーフィング：DAOリファクタリング



【演習 5-1】 DAOへの分離

既存の main メソッド内にあるJDBCのコードを、すべて新規作成した UserDAO.findUserId メソッドに移動・隠蔽せよ。

main 側は `User user = dao.findUserId("U001");` と呼び出すだけにする。

【演習 5-2】 PreparedStatementへの書き換え

SQL文の中にある `+ user_id +` のような文字列連結をすべて消滅させ、`?` に置き換えよ。setString を使って堅牢な処理を実装せよ。

© NotebookLM

- 直書き JDBC を DAO クラスに移し、呼び出し側は DAO を「1行呼ぶ」状態にする
- SQL は PreparedStatement にし、文字列連結をやめて「?」へ setXxx で値を渡す

10. 例外処理（第3回前に押さえる）

カリキュラム上は第3回の直前に、Unit90_Exception（および curriculum/units/optional-exception-handling.md）で try / catch / try-with-resources を扱う想定です。

この教材では、DAO 内で SQLException を捕捉し、まずは `e.printStackTrace()` で原因の見える化をします（本来はログ設計・通知・リカバリーを検討します）。第4回の rollback も、この例外の流れを押さえると読みやすくなります。